University of Pretoria · Department of Computer Science

COS750 Educational Software Development

COS212 DFS Tutor

Depth-First Search on a Binary Search Tree

Software Requirements and Architecture Document (SE Document)

Version	1.1
Date	28 October 2026
Status	Proposed baseline, awaiting team approval
Group	_____

Revision history

Version	Date	Change
1.0	28 Oct 2026	First draft. Algorithm was BST search (follow the ordering rule left or right).
1.1	28 Oct 2026	Algorithm changed to depth-first search (DFS). Added a plain-language explanation of DFS, an educational design summary, risks and dated plan. All UML diagrams redrawn.

Contents

1 Introduction **3**

1.1 Purpose of this document . . . 3
1.2 What we are building . . . 3
1.3 The problem we are solving . . . 3
1.4 Choice of data structure and algorithm . . . 3
1.5 Project objectives . . . 4
1.6 Scope of the two-week prototype . . . 4

2 Depth-first search in plain words **5**

2.1 The idea . . . 5
2.2 How DFS remembers where to go back to: the stack . . . 5
2.3 A worked example . . . 5
2.4 Three orders, one walk . . . 6
2.5 What DFS does *not* do, and why that matters for teaching . . . 6

3 Educational design summary **7**

3.1 Learning outcomes . . . 7
3.2 Instructional design approach: Gagné's Nine Events . . . 7
3.3 Learning theories in practice . . . 8
3.4 Catering for different learners . . . 9
3.5 Motivation (ARCS) . . . 9
3.6 Assessment and scaffolding . . . 10
3.7 Use of generative AI . . . 10

4 Technology stack **11**

5 Requirements **11**

5.1 Functional requirements . . . 11
5.2 Non-functional requirements . . . 12
5.3 Business rules . . . 13

6 System architecture **13**

6.1 Architecture decision . . . 13
6.2 Architecture diagram . . . 14

7 UML models **14**

7.1 Use case diagram . . . 14
7.2 Sequence diagram: DFS search for a value . . . 15
7.3 Activity diagram: the DFS search algorithm . . . 16

7.4 Class diagram 17
7.5 Deployment diagram 18

8 Backend API **18**
8.1 Successful search 18
8.2 Unsuccessful search 19
8.3 Traversal 19
8.4 Error handling 19

9 Core algorithm **19**

10 Project structure **20**

11 Testing and acceptance **21**
11.1 Unit tests (JUnit 5) 21
11.2 Acceptance criteria 21
11.3 Peer evaluation 22

12 Development plan **22**

13 Risks **23**

14 Decisions for the team to approve **23**

1 Introduction

1.1 Purpose of this document

This is the Software Engineering (SE) document for our COS750 examination project. It records what we are building, why, and how the pieces fit together, so that everyone in the group can work from the same picture. The instructional design itself (learning theories, motivation, assessment strategy, GenAI acknowledgement) is written up in full in the separate ID document, as the examination specification requires. Section 3 gives a short summary of it here because several software decisions only make sense once you know the teaching goal behind them.

1.2 What we are building

The COS212 lecturer has asked for an app that students can use *before* a lecture, in the way a tutor might sit with them for twenty minutes beforehand. We have chosen one data structure, the **binary search tree (BST)**, and one algorithm that runs on it, **depth-first search (DFS)**.

A student opens the app in a browser, reads a short introduction, and is shown a small tree. They can ask the app to walk the tree depth-first, either to list every value (a traversal) or to look for a particular value (a search). The app does not just print the answer. It plays the walk back one step at a time: which node is being visited, what is sitting on the stack, and why the next node is the one it is. Along the way the student is asked to predict the next move, and at the end they take a short readiness quiz. If the quiz shows they are not comfortable yet, the app gives them a guided review before they try again.

We are aiming for a working prototype in two weeks. That is not much time, so the plan keeps the technology familiar to the whole group and puts the effort where it matters for learning: the step-by-step visualisation and the feedback.

1.3 The problem we are solving

Students can usually recite "DFS goes deep first", but in our experience (and in the literature on algorithm visualisation) they struggle when asked to *apply* it. The common difficulties are:

- **Confusing depth-first with level-by-level.** After visiting the root they jump to its other child, which is breadth-first behaviour.
- **Not seeing the stack.** DFS relies on remembering where to come back to. When this is hidden inside recursion, the "backtracking" looks like magic.
- **Mixing up pre-, in- and post-order.** All three are depth-first. They visit the nodes in the same walk and differ only in *when* a node is recorded.
- **Assuming DFS uses the BST ordering.** On a BST it is tempting to think the search will "go left because the value is smaller". Plain DFS does not do that. It explores blindly, which is exactly why it is a useful contrast to set up the lecture on BST search.

The app is designed around these four misconceptions. Each one has a matching explanation, visual cue and quiz question.

1.4 Choice of data structure and algorithm

The examination specification asks for one core data structure from the CS2023 Foundational Data Structures and Algorithms knowledge unit (excluding one-dimensional arrays and linked

lists) and one core algorithm that can be applied to it [1]. Binary search trees are listed among the foundational data structures, and depth- and breadth-first traversals are listed among the core graph and tree algorithms. A BST is a good host for DFS for three reasons:

1. A binary tree is the simplest structure on which DFS has real choices to make (left or right, go deeper or back up), so the algorithm is visible without the noise of a general graph (no cycles, no visited set needed).
2. An in-order DFS of a BST produces the values in sorted order. This is a satisfying "aha" moment that links the algorithm back to the data structure.
3. Because DFS ignores the ordering, students can see for themselves that it checks more nodes than necessary. That question ("could we do better?") is a natural hand-over to the lecture.

1.5 Project objectives

When the prototype is finished, a student should be able to use it to:

1. Learn what a binary search tree is, assuming no prior knowledge.
2. Understand the ordering rule (smaller values left, larger values right).
3. See a tree drawn clearly on screen and add values to it.
4. Run a depth-first traversal in pre-order, in-order or post-order and follow it step by step.
5. Run a depth-first search for a value and see whether it was found and how many nodes were visited.
6. See the stack change at every step, so that backtracking makes sense.
7. Test their understanding with a short quiz and get feedback that explains mistakes.
8. Receive extra scaffolding (a guided review) when the quiz shows they need it.

1.6 Scope of the two-week prototype

In scope	**Out of scope (first version)**
Short introduction to trees, BSTs and DFS	User accounts and authentication
A predefined example tree	Database and saved progress
Inserting integers into the tree	Lecturer dashboard and analytics
DFS traversal (pre-, in-, post-order)	AI-powered tutoring
DFS search for a target value	Breadth-first search and other algorithms
Step player with stack view and explanations	Deletion and self-balancing trees
"Predict the next node" practice	Complex animation and gamification
Readiness quiz with feedback and guided review	Mobile-first layout
Reset to the example tree	

These limits give us a target we can actually hit, with time left for testing, documentation and rehearsing the demonstration.

2 Depth-first search in plain words

This section is written for the team, but the wording is deliberately close to what students will see in the app. If we can explain it simply here, we can explain it simply on screen.

2.1 The idea

Imagine you are exploring a maze. At every fork you take the left-hand passage and keep going until you hit a dead end. Then you walk back to the most recent fork where there is still a passage you have not tried, and you take that one. You carry on like this until you have found what you are looking for or there is nowhere left to go.

That is depth-first search. "Depth-first" simply means: *go as deep as you can down one path before trying another*. Walking back to the last unexplored fork is called **backtracking**.

In a tree, the forks are the nodes and the passages are the left and right children. A leaf is a dead end.

2.2 How DFS remembers where to go back to: the stack

To backtrack, you need to remember the forks you have not finished with. DFS keeps these in a **stack**, which works like a pile of plates: you can only add a plate to the top, and you can only take the top plate off. The last thing you put on is the first thing you take off (last in, first out).

The DFS search used in the app follows four simple steps:

1. Put the root on the stack.
2. Take the top node off the stack and *visit* it (compare it with the target).
3. If it is the target, stop: found. Otherwise put its right child on the stack, then its left child (skip any that are empty). Pushing right first means the left child ends up on top, so left is explored first.
4. If the stack is empty, stop: the value is not in the tree. Otherwise go back to step 2.

2.3 A worked example

Figure 1 shows the example tree the app opens with. The small orange numbers give the order in which DFS visits the nodes (pre-order).

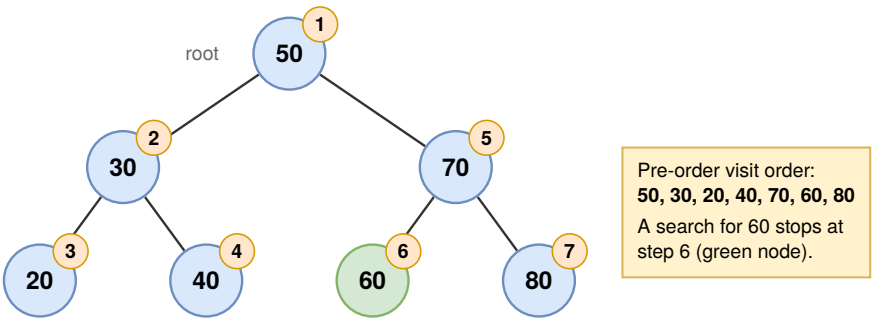


Figure 1: The example BST. Badges show the order in which a pre-order DFS visits each node.

Table 1 traces a search for 60. The stack is written with the top on the left.

Step	Visit	Is it 60?	What happens	Stack after (top first)
0	—	—	Start: push the root	50
1	50	no	Push right child 70, then left child 30	30, 70
2	30	no	Push 40, then 20	20, 40, 70
3	20	no	Leaf, nothing to push. Backtrack.	40, 70
4	40	no	Leaf, nothing to push. Backtrack.	70
5	70	no	Push 80, then 60	60, 80
6	60	**yes**	Found. Stop.	80

Table 1: DFS search for 60. Six nodes are visited.

Figure 2 shows the same thing the way the app's stack panel will draw it.

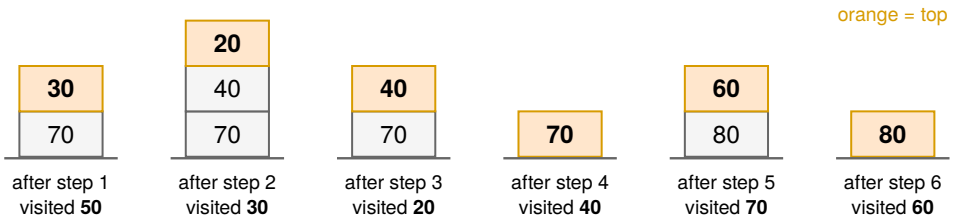


Figure 2: Stack contents after each step of the search for 60. The orange box is the next node to be taken off.

If we search for a value that is not in the tree, say 65, DFS has no way of knowing it can give up early. It visits all seven nodes before the stack runs empty and it reports "not found".

2.4 Three orders, one walk

DFS can also be used to list every value in the tree. The walk through the tree is identical in each case; the only difference is *when* we write a node down:

Order	When the node is recorded	Result for the example tree
Pre-order	On the way down, before its children	50, 30, 20, 40, 70, 60, 80
In-order	Between its left and right subtree	20, 30, 40, 50, 60, 70, 80
Post-order	On the way back up, after its children	20, 40, 30, 60, 80, 70, 50

Table 2: The three depth-first orders on the example tree.

Notice that the in-order result is sorted. This is not a coincidence: everything to the left of a node is smaller and everything to the right is larger, so "left, me, right" produces values from smallest to largest. The app points this out explicitly because it is the moment where the algorithm and the data structure meet.

2.5 What DFS does *not* do, and why that matters for teaching

DFS treats the tree as a plain tree. It does not look at the values to decide where to go. On our example, finding 60 takes six visits, whereas a search that uses the ordering rule ("60 is bigger than 50, go right; smaller than 70, go left") would need only three. We show this comparison on

the summary screen as an open question for the student to bring to the lecture. It is a deliberate cliff-hanger rather than a gap in the app.

3 Educational design summary

The full instructional design is in the ID document. This summary is here so that the requirements in Section 5 can be traced back to a teaching reason.

3.1 Learning outcomes

The outcomes are written against the revised Bloom's taxonomy [2] so that each quiz question can be tied to a level.

ID	Bloom level	By the end of the session, the student can...
LO1	Remember	state the BST ordering rule and name the data structure DFS uses (a stack).
LO2	Understand	explain in their own words what "depth-first" and "backtracking" mean.
LO3	Apply	write down the pre-, in- and post-order of a small BST.
LO4	Apply	trace a DFS search for a value, giving the stack after each step.
LO5	Analyse	predict how many nodes DFS will visit for a given target, and explain why the in-order of a BST is sorted.

3.2 Instructional design approach: Gagné's Nine Events

We use Gagné's Nine Events of Instruction [3] to sequence the screens. It suits a short, self-paced pre-lecture session because it moves from attention and prior knowledge, through guided examples, to practice, feedback and assessment, all in one sitting. ADDIE is used as the overall project process (analyse, design, develop, implement, evaluate) and lines up with the development plan in Section 12.

Gagné event	What the app does	Screen / feature
1 Gain attention	Short maze puzzle: "Which way would you explore first?"	Hook screen
2 Inform of objectives	Three plain-language goals for the session	Goals card
3 Recall prior learning	Two quick questions on trees and stacks; links to a refresher if needed	Warm-up check
4 Present the content	Plain-language DFS explanation with the maze analogy	Concept screens
5 Provide guidance	Worked example played step by step with narration	Step player
6 Elicit performance	Student predicts the next node before it is revealed	Predict mode
7 Provide feedback	Immediate, specific feedback on every prediction and answer	Feedback panel
8 Assess performance	Six-question readiness quiz	Quiz
9 Enhance retention and transfer	Summary, "try your own tree", questions for the lecture	Summary screen

Figure 3 shows how a student moves through these screens, including the remediation loop.

3.3 Learning theories in practice

Cognitive load theory [4].
DFS has a lot going on at once: the tree, the current node, the stack and the order. We reduce load by *segmenting* (one step per click), *signalling* (colour for current, visited, on-stack and found nodes) and keeping explanation text right next to the diagram rather than in a separate panel. The student controls the pace; there is no auto-play that runs ahead of them.

Worked examples and fading.
The first example is fully worked. In the second the student predicts the next node and the app confirms. In the third they trace on their own. This follows the worked-example effect and its fading strategy, which is well suited to novices.

Active learning and engagement.
Research on algorithm visualisation shows that *watching* an animation adds little; learning improves when students have to respond to it or change it [5, 6]. "Predict the next node" puts students at the *responding* level of the engagement taxonomy, and inserting their own values puts them at the *changing* level.

Scaffolding and the zone of proximal development [7].
Hints are layered: a nudge first ("Which node is on top of the stack?"), then a stronger hint, then the answer with an explanation. The guided review is the app acting as the "more knowledgeable other" for students who are not ready yet.

Dual coding and multiple representations [8].
Every step is shown as a picture (the tree), a structure (the stack), words (the explanation) and, optionally, code (the highlighted line in the Java method).

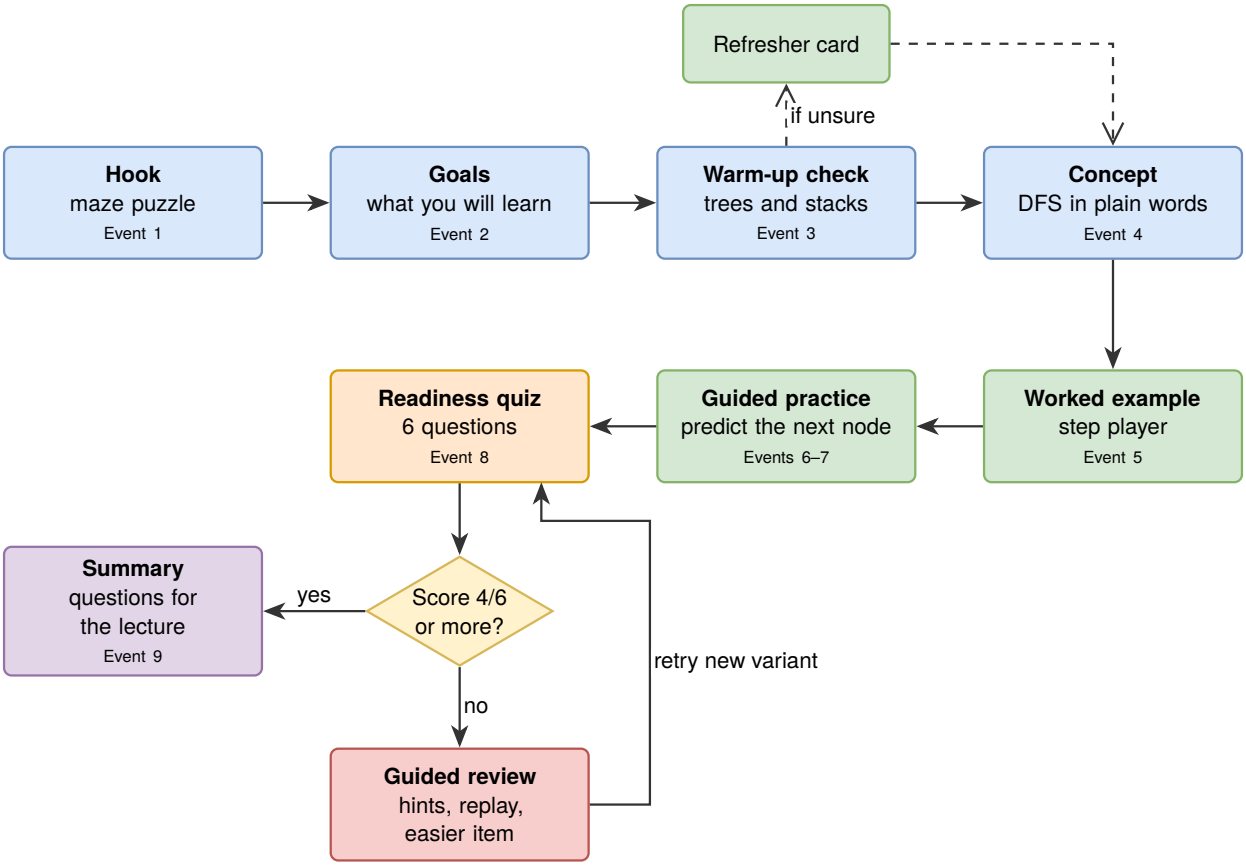


Figure 3: Student learning journey. Blue: instruction; green: guided practice; orange: assessment; red: remediation; purple: consolidation.

3.4 Catering for different learners

The specification asks which learning styles the app caters for. We map our features onto the VARK categories because they are familiar, but we are careful not to over-claim: there is little evidence that matching teaching to a preferred style improves results [9]. Our approach is therefore to give *every* student several representations, which helps all learners regardless of preference.

VARK category	How the app supports it
Visual	Tree diagram, colour-coded states, animated stack panel.
Aural	Optional read-aloud of step explanations using the browser's built-in speech synthesis ("Could" priority).
Read/write	Short text explanations, a glossary, the Java code view.
Kinaesthetic	Stepping manually, predicting, inserting values and building their own tree.

Accessibility is treated as a requirement, not an extra: node states are never shown by colour alone (each also has an icon or outline style), all controls work from the keyboard, and text meets WCAG AA contrast.

3.5 Motivation (ARCS)

We use Keller's ARCS model [10] to keep students engaged for the twenty minutes or so a session takes.

ARCS	Design choice
Attention	Maze hook; predict-then-reveal creates small moments of suspense.
Relevance	Links to things students use: folder trees, undo history, and the COS212 practicals.
Confidence	Clear goals, short steps, early easy wins, and a progress bar showing where they are.
Satisfaction	Immediate feedback, the "sorted in-order" discovery, and a summary of what they now know.

3.6 Assessment and scaffolding

Assessment is formative. Its job is to tell the student (not the lecturer) whether they are ready for the lecture, and to help them if they are not. Each question is tied to a learning outcome and wrong options are built from the misconceptions listed in Section 1.3, so the feedback can say *why* an answer is wrong, not only that it is wrong.

Q	Question (example tree)	LO	Feedback on the typical wrong answer
1	Which structure does DFS use to remember where to go back to?	LO1	*Queue*: "A queue serves the oldest item first, which explores level by level. That is breadth-first search. DFS needs the newest item first: a stack."
2	DFS has just visited 30. Which node is visited next?	LO2	*70*: "70 is on the same level as 30. Depth-first goes *down* before across, so the next node is 30's left child, 20."
3	Give the pre-order of the tree.	LO3	*20, 30, 40, …*: "That is in-order. In pre-order a node is written down *before* its children, so the root 50 comes first."
4	The stack is [40, 70] (top first). What is popped next?	LO4	*70*: "The top of the stack is on the left. The last node pushed was 40, so 40 comes off first."
5	How many nodes does DFS visit to find 80?	LO5	*3*: "Three would be right if DFS used the ordering rule. Plain DFS explores the whole left subtree first, so it visits all 7 nodes."
6	Why is the in-order of a BST always sorted?	LO5	Explanation built from "left is smaller, right is larger"; a diagram highlights each subtree.

Scaffolding rule. A student who scores 4/6 or more goes to the summary screen. Below that, the app opens a guided review that targets the outcomes they missed: it replays the relevant part of the worked example, asks an easier version of the question with a hint, and then offers a fresh variant of the quiz (different tree, same outcomes). The student can always choose to continue to the summary; the review is offered, not forced.

3.7 Use of generative AI

The specification allows GenAI for content and GUI work, provided it is acknowledged. Any prompts used to draft lesson text, quiz items or interface code, and the tool used, will be recorded in an appendix of the ID document. All generated quiz items are checked by two group members against the reference Java implementation before they go into the app.

4 Technology stack

The backend is Java, so the simplest path is a single Spring Boot application that serves both the REST API and a plain HTML, CSS and JavaScript frontend.

Component	Technology	Purpose
Backend	Java 21, Spring Boot 3	BST and DFS logic; REST endpoints.
Frontend	HTML, CSS, JavaScript (no framework)	Lesson screens, step player, quiz.
API	REST with JSON	Browser to backend communication.
Tree visualisation	SVG generated in JavaScript	Draws nodes, edges and highlights.
Build	Maven	Dependencies, compilation, tests.
Unit testing	JUnit 5	Tests for insertion, DFS orders and search.
Version control	Git and GitHub	Issues, pull requests, code review.
Deployment	Local machine (`localhost:8080`)	Simple to run for the demonstration.

Recommended decision. Put the frontend in `src/main/resources/static` so Spring Boot serves it. One command (`mvn spring-boot:run`) then starts the whole prototype, with no separate frontend server and no database. The browser calls the backend with relative URLs such as `/api/dfs/search`, which means nothing has to change if we later host it elsewhere.

If some members already have Java 17 installed and working, Java 17 with a compatible Spring Boot 3 release is fine. Whatever we choose, we pin it in `pom.xml` so the build is the same on every machine.

5 Requirements

5.1 Functional requirements

Priorities use MoSCoW: **M**ust, **S**hould, **C**ould.

ID	Requirement	Pri.	Trace
FR-01	Show an introduction to trees, BSTs and DFS that assumes no prior knowledge.	M	LO1, LO2
FR-02	Show the predefined example BST when the student opens the explorer.	M	LO3
FR-03	Let the student insert an integer into the tree, following the BST rule.	M	LO1
FR-04	Let the student choose a DFS order (pre-, in- or post-order) and run a traversal.	M	LO3
FR-05	Let the student enter a target value and run a DFS search (pre-order).	M	LO4
FR-06	Play the DFS back one step at a time, with next, previous and restart controls.	M	LO2, LO4

ID	Requirement	Pri.	Trace
FR-07	At each step, highlight the current node, mark visited nodes and show the stack.	M	LO4
FR-08	At each step, show a one-sentence explanation of what happened and why.	M	LO2
FR-09	After a search, show whether the target was found and how many nodes were visited.	M	LO5
FR-10	Offer a "predict the next node" mode with immediate feedback.	M	LO4
FR-11	Provide a readiness quiz linked to the learning outcomes.	M	all
FR-12	Give specific feedback that explains both correct and incorrect answers.	M	all
FR-13	Offer a guided review when the quiz score is below the threshold.	M	all
FR-14	Let the student reset the tree to the example.	S	—
FR-15	Show a skewed tree (values inserted in sorted order) and how DFS behaves on it.	S	LO5
FR-16	Show the matching line of Java code alongside each step.	S	LO2
FR-17	Read step explanations aloud on request.	C	—

5.2 Non-functional requirements

ID	Requirement
NFR-01	**Usability.** A first-year student can complete the session without help. Checked with at least three peer testers before the demo.
NFR-02	**Responsiveness.** Results appear within 300 ms on a laptop for trees of up to 15 nodes.
NFR-03	**Correctness.** The steps shown in the browser come from the Java algorithm, never from a separate JavaScript re-implementation, so the two cannot disagree.
NFR-04	**Error messages.** Invalid input produces a short, friendly message that says what to do next. No stack traces are shown to students.
NFR-05	**Compatibility.** Works in current Chrome, Firefox, Edge and Safari on a laptop screen.
NFR-06	**Accessibility.** Keyboard operable; colour is never the only signal; WCAG AA contrast.
NFR-07	**Maintainability.** Java classes have one clear responsibility each; the BST and DFS classes do not depend on Spring.
NFR-08	**Testability.** BST insertion and every DFS order have automated JUnit tests.
NFR-09	**Portability.** The app runs locally from the README instructions with one command.
NFR-10	**Pacing.** The student controls every step; nothing advances without their action.

5.3 Business rules

Tree rules

BR-1 Every node holds a whole number from 0 to 99 (two digits keep the circles readable).

BR-2 A value smaller than a node goes into that node's left subtree.

BR-3 A value larger than a node goes into that node's right subtree.

BR-4 Duplicate values are rejected with an explanation.

BR-5 A tree holds at most 15 nodes so that it fits on a laptop screen.

DFS rules

BR-6 Every DFS starts at the root.

BR-7 DFS goes as deep as possible along one branch before backtracking.

BR-8 The left child is always explored before the right child (matches the COS212 convention).

BR-9 DFS search checks each node when it is first reached (pre-order) and stops as soon as the target is found or the stack is empty.

BR-10 "Nodes visited" counts only real nodes, never empty (null) children. Each node is visited at most once.

BR-11 DFS does *not* use the ordering of the BST to skip subtrees. This is intentional: it is what makes it DFS rather than BST search.

6 System architecture

6.1 Architecture decision

We use a simple layered architecture inside one Spring Boot application:

- The **browser** handles everything the student sees: lesson screens, the SVG tree, the step player and the quiz.
- A **controller** layer turns HTTP requests into method calls and results into JSON.
- A **service** layer holds the current tree and coordinates the work.
- A **domain** layer contains the BST and the DFS algorithm as plain Java, with no Spring code, so they can be unit-tested directly.

The algorithm is kept separate from the data structure (`DepthFirstSearch` operates on a `BinarySearchTree` rather than being a method inside it). This mirrors how we want students to think: the tree is one idea, the walk over it is another.

There is no database. The tree lives in memory and the student can reset it at any time. The quiz runs entirely in JavaScript from a fixed question bank, because it does not need anything from the server.

6.2 Architecture diagram

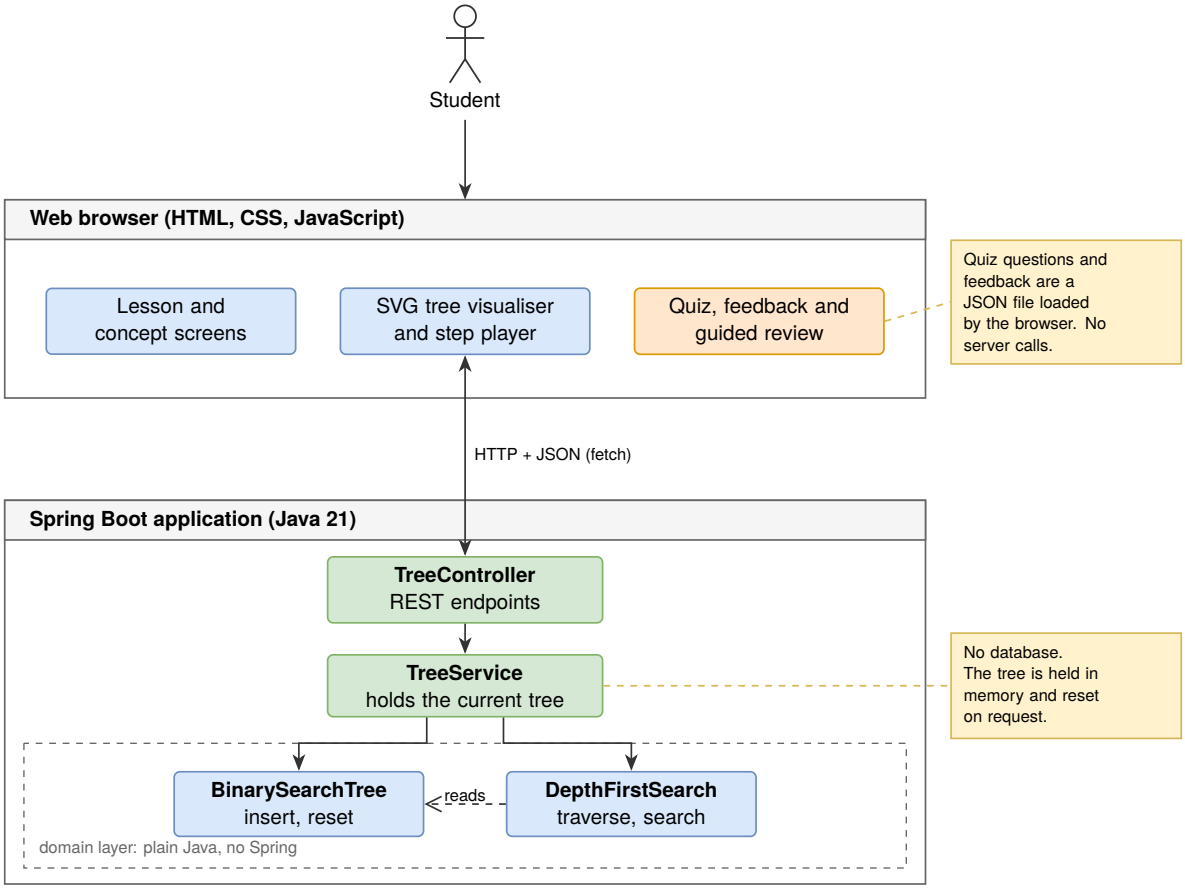


Figure 4: Layered architecture of the COS212 DFS Tutor.

How a request flows

1. The student opens `http://localhost:8080` and the browser loads the static files.
2. The student chooses an action, for example “Search for 60”.
3. JavaScript checks the input, then sends a request to the Spring Boot backend.
4. The controller passes the request to the service.
5. The service asks `DepthFirstSearch` to run over the current `BinarySearchTree`.
6. The algorithm records every step (visited node, stack contents, explanation) and returns them.
7. The backend sends the result back as JSON.
8. The browser plays the steps back one at a time as the student clicks “Next”.

Returning all the steps in one response, rather than calling the server once per step, keeps the step player fast and lets the student go backwards as well as forwards.

7 UML models

7.1 Use case diagram

The only actor is the student. There is no lecturer or administrator role in the prototype.

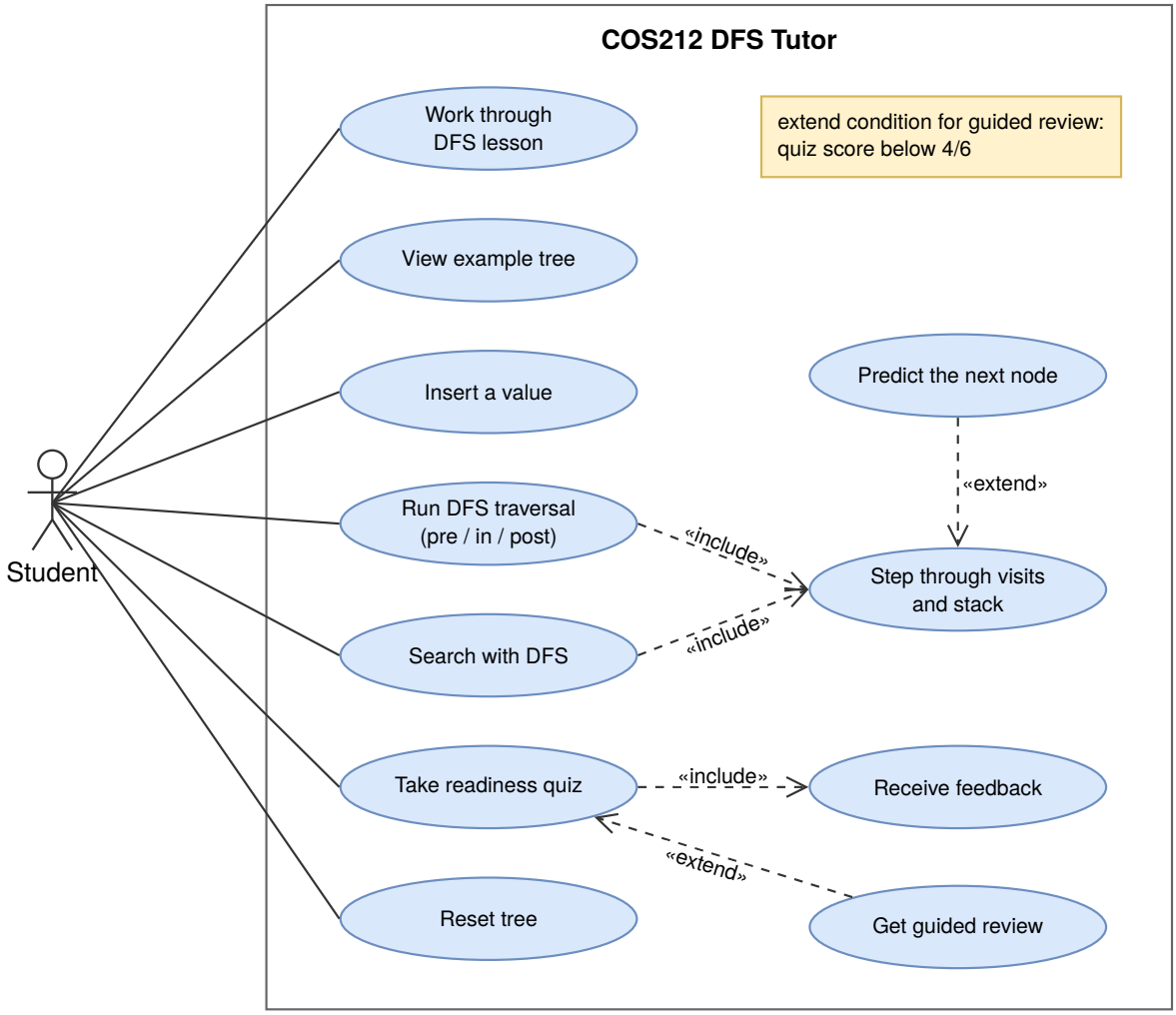


Figure 5: Use case diagram.

7.2 Sequence diagram: DFS search for a value

This shows what happens between the browser and the backend classes when a student searches for 60.

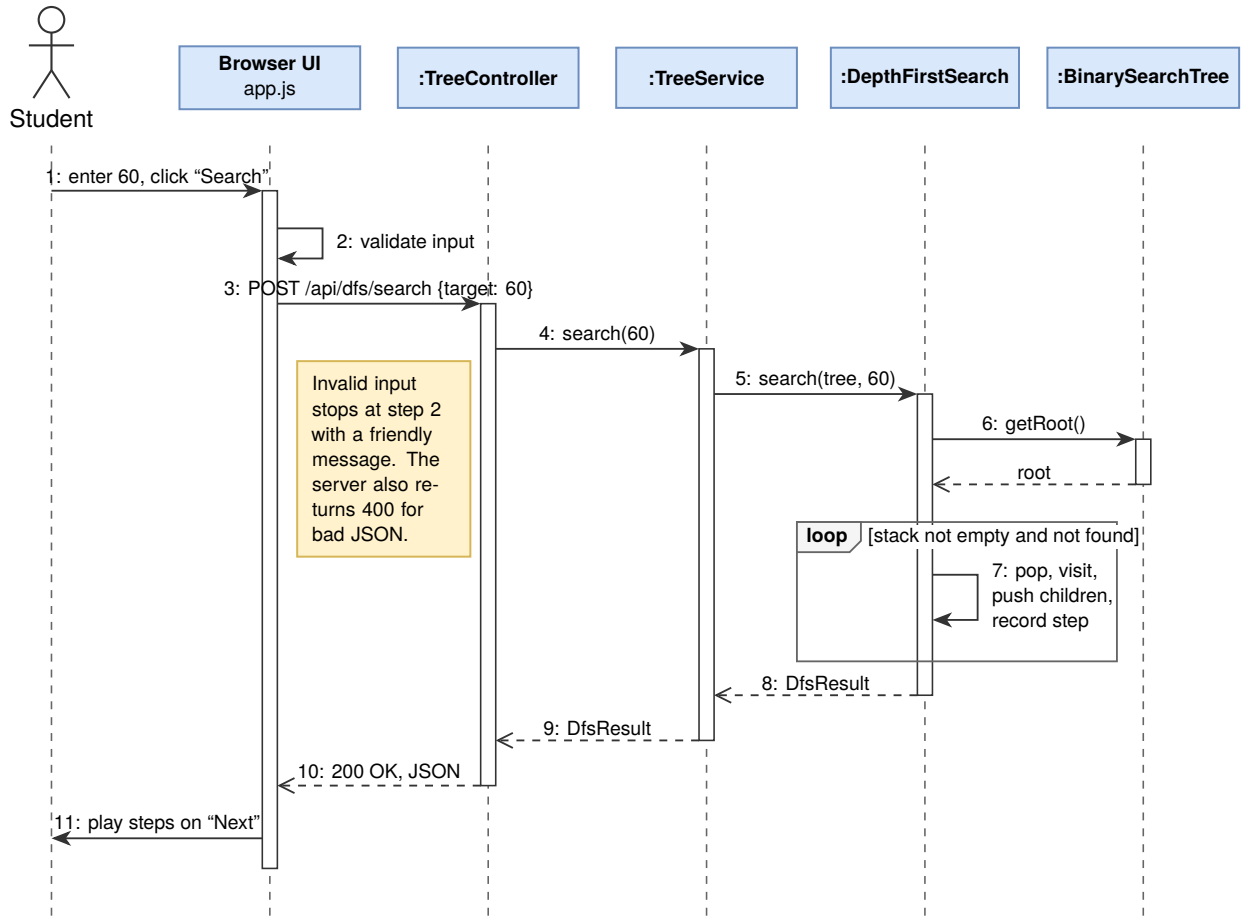


Figure 6: Sequence diagram for a DFS search.

7.3 Activity diagram: the DFS search algorithm

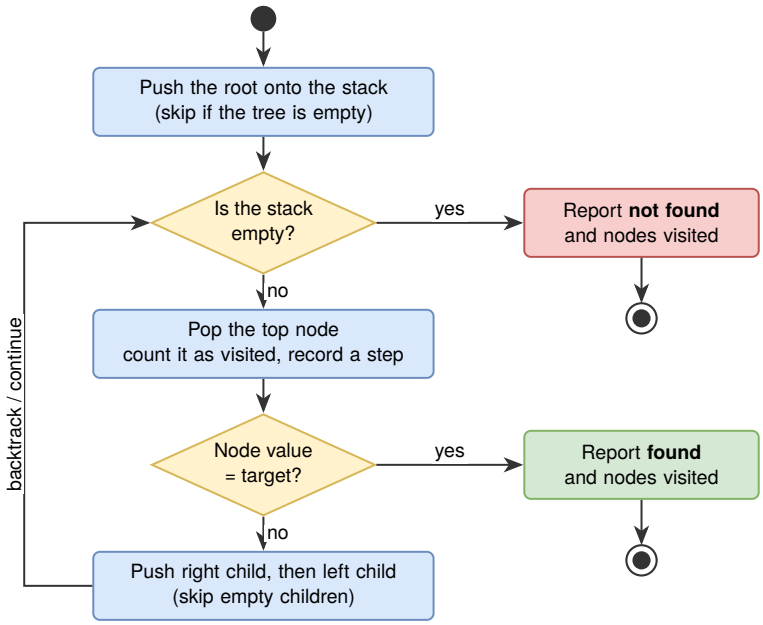


Figure 7: Activity diagram for the iterative DFS search used in the app.

7.4 Class diagram

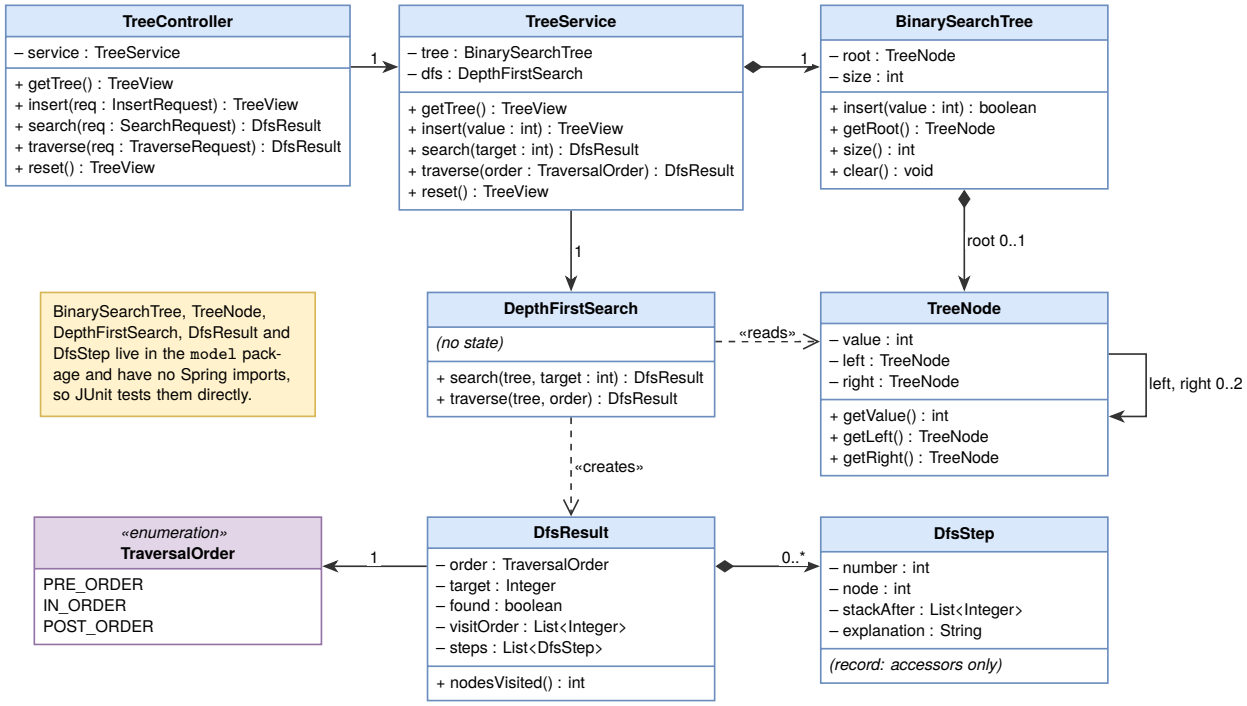


Figure 8: Class diagram for the Java backend (DTOs such as `TreeView` and the request records are omitted for clarity).

Responsibilities

- `TreeNode`: stores a value and references to its left and right children.
- `BinarySearchTree`: builds the tree by the BST rule; rejects duplicates; can be cleared.
- `DepthFirstSearch`: runs DFS (search or traversal) over a tree and records every step. It never changes the tree.
- `DfsResult` and `DfsStep`: immutable records holding the outcome, the visit order and the step-by-step history used by the step player.
- `TraversalOrder`: the three depth-first orders.
- `TreeService`: owns the current tree, seeds the example tree, enforces the 15-node limit.
- `TreeController`: exposes the HTTP endpoints and turns errors into friendly 400 responses.
- `TreeView` (DTO): a nested JSON-friendly copy of the tree for the visualiser.

7.5 Deployment diagram

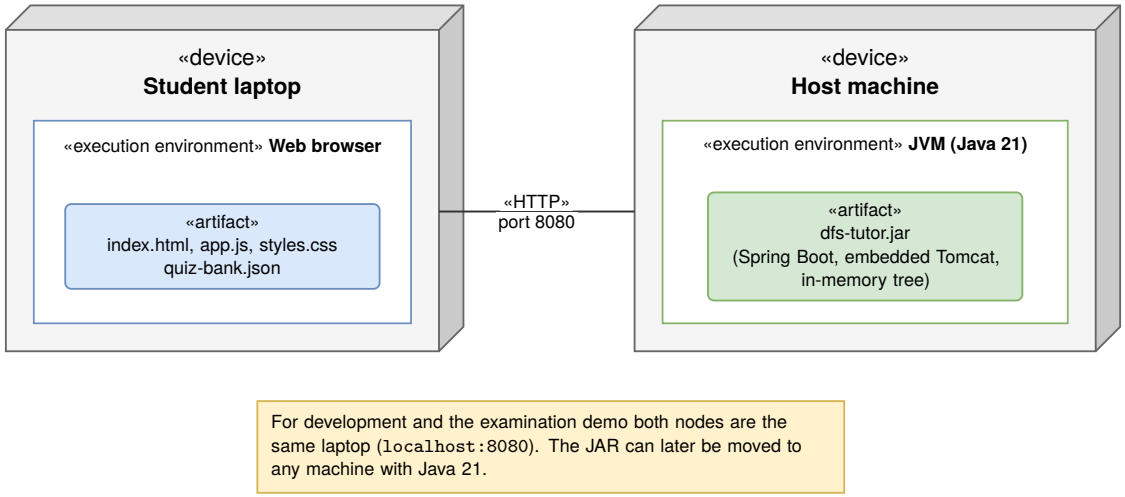


Figure 9: Deployment diagram.

8 Backend API

Method	Endpoint	Purpose
`GET`	`/api/tree`	Return the current tree.
`POST`	`/api/tree/insert`	Insert an integer: `{"value": 65}`.
`POST`	`/api/tree/reset`	Restore the example tree.
`POST`	`/api/dfs/search`	DFS search: `{"target": 60}`.
`POST`	`/api/dfs/traverse`	DFS traversal: `{"order": "IN_ORDER"}`.

8.1 Successful search

`POST /api/dfs/search` with body `{"target": 60}` returns:

```
{
  "order": "PRE_ORDER",
  "target": 60,
  "found": true,
  "visitOrder": [50, 30, 20, 40, 70, 60],
  "nodesVisited": 6,
  "steps": [
    { "number": 1, "node": 50, "stackAfter": [30, 70],
      "explanation": "Visit 50. It is not 60. Push 70 (right), then 30 (left), so 30 is on top." },

    { "number": 2, "node": 30, "stackAfter": [20, 40, 70],
      "explanation": "Visit 30. Not 60. Push 40, then 20." },
    ...
    { "number": 6, "node": 60, "stackAfter": [80],
      "explanation": "Visit 60. This is the target. Stop searching." }
  ]
}
```

`stackAfter` lists the stack top first. The browser uses `steps` to drive the step player and the stack panel, so it never has to work out the algorithm itself (NFR-03).

8.2 Unsuccessful search

```
{
  "order": "PRE_ORDER",
  "target": 65,
  "found": false,
  "visitOrder": [50, 30, 20, 40, 70, 60, 80],
  "nodesVisited": 7,
  "steps": [ ... ]
}
```

8.3 Traversal

`POST /api/dfs/traverse` with `{"order": "IN_ORDER"}` returns the same shape with `target` set to `null`, `found` set to `false` and `"visitOrder": [20, 30, 40, 50, 60, 70, 80]`. For traversals `stackAfter` holds the recursion (call) stack, which is the path from the root to the current node.

8.4 Error handling

All errors use HTTP `400 Bad Request` (or `500` for unexpected faults) and a body the frontend can show directly:

```
{ "error": "DUPLICATE_VALUE",
  "message": "40 is already in the tree. Each value can appear only once in this BST." }
```

Error code	When
`MISSING_VALUE`	Target or value is missing.
`INVALID_NUMBER`	Not a whole number, or outside 0–99.
`INVALID_ORDER`	Order is not one of the three allowed values.
`DUPLICATE_VALUE`	Inserting a value that already exists.
`TREE_FULL`	Inserting a sixteenth node.
`INTERNAL_ERROR`	Anything unexpected. The message is generic; details go to the server log.

9 Core algorithm

The search is written iteratively with an explicit stack, because the stack is exactly what we want students to see. The traversal is written recursively, because it shows very compactly that the three orders differ only in where one line sits.

Listing 1: DFS search (iterative, pre-order).

```
1  public DfsResult search(BinarySearchTree tree, int target) {
2      DfsResult.Builder result = DfsResult.builder(TraversalOrder.PRE_ORDER, target);
3      Deque<TreeNode> stack = new ArrayDeque<>();
4      if (tree.getRoot() != null) {
5          stack.push(tree.getRoot());
6      }
7      while (!stack.isEmpty()) {
8          TreeNode current = stack.pop();
9          if (current.getValue() == target) {
10             result.visit(current.getValue(), stack, "This is the target. Stop searching.");
11             return result.found();
12         }
```

```
        // Push right first so that left ends up on top and is explored first.
        if (current.getRight() != null) stack.push(current.getRight());
        if (current.getLeft() != null)  stack.push(current.getLeft());
        result.visit(current.getValue(), stack, explain(current));
    }
    return result.notFound();
}
```

Listing 2: DFS traversal (recursive). Only the position of `visit` changes between orders.

```
private void walk(TreeNode node, TraversalOrder order,
                  Deque<Integer> path, DfsResult.Builder result) {
    if (node == null) return;
    path.push(node.getValue());
    if (order == TraversalOrder.PRE_ORDER)  result.visit(node.getValue(), path);
    walk(node.getLeft(), order, path, result);
    if (order == TraversalOrder.IN_ORDER)   result.visit(node.getValue(), path);
    walk(node.getRight(), order, path, result);
    if (order == TraversalOrder.POST_ORDER) result.visit(node.getValue(), path);
    path.pop();
}
```

Both run in $O(n)$ time for a tree of n nodes, since each node is visited at most once. The extra space is $O(h)$, where h is the height of the tree: about $\log_2 n$ for a balanced tree and up to n for a skewed one. FR-15 lets students see this difference.

10 Project structure

```
cos212-dfs-tutor/
|-- pom.xml
|-- README.md
|-- docs/
|   |-- SE-document.tex          (this document)
|   `-- ID-document.tex
`-- src/
    |-- main/
    |   |-- java/za/ac/up/dfstutor/
    |   |   |-- DfsTutorApplication.java
    |   |   |-- controller/  TreeController.java, ApiExceptionHandler.java
    |   |   |-- service/     TreeService.java
    |   |   |-- model/       TreeNode.java, BinarySearchTree.java,
    |   |   |                DepthFirstSearch.java, TraversalOrder.java,
    |   |   |                DfsResult.java, DfsStep.java
    |   |   `-- dto/         InsertRequest.java, SearchRequest.java,
    |   |                    TraverseRequest.java, TreeView.java, ApiError.java
    |   `-- resources/
    |       |-- application.properties
    |       `-- static/
    |           |-- index.html
    |           |-- css/styles.css
    |           |-- js/app.js, js/tree-view.js, js/step-player.js, js/quiz.js
    |           `-- data/quiz-bank.json, data/lesson.json
    `-- test/java/za/ac/up/dfstutor/
        |-- BinarySearchTreeTest.java
        |-- DepthFirstSearchTest.java
        `-- TreeControllerTest.java
```

This is enough for the first version. We avoid extra interfaces, factories and repositories unless the code gives us a concrete reason to add them. Lesson text and quiz questions live in JSON files so that the ID team can edit content without touching JavaScript.

11 Testing and acceptance

11.1 Unit tests (JUnit 5)

- Empty tree: search returns not found with 0 nodes visited.
- Root as target: found after 1 visit.
- Search for 60 on the example tree: visit order is exactly 50, 30, 20, 40, 70, 60.
- Search for 65: not found, all 7 nodes visited.
- Pre-, in- and post-order on the example tree match Table 2.
- In-order of any tree built from random distinct values is sorted (property-style test).
- `stackAfter` for every step of the 60 search matches Table 1.
- Skewed tree (10, 20, 30, 40): every order is correct and the stack never holds more than one pending right child.
- Inserting a duplicate returns `false` and leaves the tree unchanged.
- Inserting a sixteenth value is rejected by the service.

11.2 Acceptance criteria

The prototype is ready to demonstrate when every check below passes.

Check	**Expected result**
Open the app	The introduction is shown, followed by the example tree.
Pre-order traversal	Visits 50, 30, 20, 40, 70, 60, 80 with the stack shown at each step.
In-order traversal	Visits 20 to 80 in sorted order; the app points out that it is sorted.
Post-order traversal	Visits 20, 40, 30, 60, 80, 70, 50.
Search for the root (50)	Found after 1 visit.
Search for 60	Found after 6 visits; the whole left subtree is explored first.
Search for a missing value (65)	"Not found" after visiting all 7 nodes.
Step backwards	The previous step, highlight and stack are restored exactly.
Predict the next node, correctly	Confirmation with a one-line reason.
Predict the next node, incorrectly	Explanation of the mistake and a hint; the student can try again.
Insert a valid integer	The node appears in the correct position.
Insert a duplicate	Message explaining that the value already exists.
Insert invalid input ("abc", 150)	Friendly validation message; nothing is sent to the server for "abc".
Reset the tree	The example tree is restored.
Quiz score 4/6 or higher	Summary screen with questions for the lecture.

Check	Expected result
Quiz score below 4/6	Guided review is offered for the missed outcomes, then a new quiz variant.
Keyboard only	Every screen can be completed without a mouse.

11.3 Peer evaluation

Between days 11 and 12, at least three students who have not taken COS212 yet (or who have forgotten DFS) use the app without help while a group member observes. We note where they hesitate, what they misread and how long a session takes. Changes are limited to fixes that remove confusion; new features are not added at this stage. A short summary of findings goes into the ID document's evaluation section.

12 Development plan

Today is Wednesday 28 October 2026. Documentation is due on ClickUP before 08:00 on Monday 16 November, and the presentation is on 16 or 17 November. The plan below finishes the prototype by 11 November and leaves four days of buffer.

Days	Dates	Work	Done when...
1–2	29–30 Oct	Approve this document; create repository; Spring Boot skeleton; start ID document outline.	App starts and shows a placeholder page.
3–4	31 Oct–1 Nov	`BinarySearchTree`, `DepthFirstSearch`, step recording, JUnit tests.	All unit tests pass.
5–6	2–3 Nov	REST endpoints and error handling; first end-to-end search from the browser.	Search works through HTTP.
7–8	4–5 Nov	SVG tree, step player, stack panel, highlights.	Students can watch DFS work.
9–10	6–7 Nov	Lesson screens, predict mode, quiz, feedback, guided review.	Full learning journey works.
11–12	8–9 Nov	Edge cases, accessibility pass, peer testing, bug fixes.	Acceptance checks pass.
13	10 Nov	Finalise SE and ID documents, README, slides.	Documents ready to submit.
14	11 Nov	Full rehearsal (10 min talk + 10 min demo); tag release `v1.0`.	Demo runs cleanly twice.
—	12–15 Nov	Buffer. Submit documents early.	—

One rule we will stick to: integrate as soon as the first search endpoint works (day 5 or 6). We do not wait for the frontend to be finished before testing it against the real Java backend.

Other deadlines. Self-reflection reports are individual and due before 20:00 on Wednesday 18 November 2026.

13 Risks

Risk	Likelihood	What we do about it
The step player takes longer than planned.	Medium	Build a plain list view of the steps first; add animation only if time allows.
Frontend and backend disagree on the JSON.	Medium	Agree the JSON examples in this document on day 2 and integrate early.
Skewed trees do not fit on screen.	Low	15-node limit; SVG scales to fit and scrolls vertically.
Quiz content is wrong or confusing.	Medium	Every item is checked against the Java output by two members.
The demo fails on the day (laptop, projector).	Low	Run locally with no internet; keep a built JAR on two laptops and a short screen recording as backup.
A member becomes unavailable.	Medium	Pair on every component so at least two people know each part.

14 Decisions for the team to approve

Before development starts, every member should agree to the following:

1. Data structure: binary search tree. Algorithm: depth-first search (search and the three traversal orders).
2. Java 21 with Spring Boot 3 (or Java 17 if that is what members already have), built with Maven.
3. Plain HTML, CSS and JavaScript served by Spring Boot; no frontend framework.
4. No database and no user accounts in the first version.
5. Values 0–99, at most 15 nodes, duplicates rejected.
6. The backend returns every DFS step; the browser only displays them.
7. Gagné's Nine Events structures the lesson; ADDIE structures the project.
8. Quiz pass threshold of 4/6, with guided review below that.
9. GitHub Issues and pull requests for all work; every pull request is reviewed by one other member.

Once these are approved, the next step is to create the GitHub repository, initialise the Spring Boot project and assign the first two tasks: the BST and DFS implementation with tests, and the basic frontend shell.

References

[1] A. N. Kumar, R. K. Raj, S. G. Aly, M. D. Anderson, B. A. Becker et al. *Computer Science Curricula 2023*. Association for Computing Machinery, New York, 2024.

[2] L. W. Anderson and D. R. Krathwohl (eds.). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, 2001.

[3] R. M. Gagné, W. W. Wager, K. C. Golas and J. M. Keller. *Principles of Instructional Design*, 5th ed. Wadsworth, 2005.

[4] J. Sweller. Cognitive load during problem solving: effects on learning. *Cognitive Science*, 12(2):257–285, 1988.

[5] C. D. Hundhausen, S. A. Douglas and J. T. Stasko. A meta-study of algorithm visualization effectiveness. *Journal of Visual Languages and Computing*, 13(3):259–290, 2002.

[6] T. L. Naps, G. Rößling et al. Exploring the role of visualization and engagement in computer science education. *ACM SIGCSE Bulletin*, 35(2):131–152, 2003 (ITiCSE 2002 working group report).

[7] L. S. Vygotsky. *Mind in Society: The Development of Higher Psychological Processes*. Harvard University Press, 1978.

[8] R. E. Mayer. *Multimedia Learning*, 2nd ed. Cambridge University Press, 2009.

[9] H. Pashler, M. McDaniel, D. Rohrer and R. Bjork. Learning styles: concepts and evidence. *Psychological Science in the Public Interest*, 9(3):105–119, 2008.

[10] J. M. Keller. Development and use of the ARCS model of instructional design. *Journal of Instructional Development*, 10(3):2–10, 1987.